

从已验证能力到合法派生

面向 Agent 的预研开发形式方法

(教材式草案 · 持续修订)

ascendc 工程 · 专题分支 `research/formal-lang-dag`

摘要

愿景：把「层层递进、先验证底层再拼高层」的工程直觉，提升为一套可教学、可门禁、可对照实验的形式方法——使 Agent（以及人类协作者）在复杂预研任务中，**只沿已验证能力及其合法组合**生长实现，从机制上压缩跳层发明与幻觉空间；最终服务一般预研开发，而非仅服务某一密码标准的算子清单。

写法：仿教材而非论文——先从特例讲故事，再抽象数学对象，再复盘与前瞻；证明与完备性后置，允许边做边补。**地位：**`docs/research/` 调研草稿，非定稿、非交付验收依据。

压力测试床：ML-KEM-1024 链上的 `pke.keygen / kem.keygen / kem.encaps`，并以 `kem.decaps` 为理论前瞻试金石；`*-correctness-*` 作为「只求正确、不计优化」的对照组标本。

目录

1 愿景、读者与本文怎么读	2
1.1 愿景（一句话）	2
1.2 我们希望最终得到什么	2
1.3 研究路线：从特殊到一般	2
1.4 阅读地图（教材顺序）	2
2 引子：从 <code>pke.keygen</code> 到 <code>kem.keygen</code>	2
2.1 先给人一张「接缝图」	2
2.2 为何这已经是「图」而不是「树」	3
2.3 引子要留下的问题	3
3 回溯：一层层剥开直到平台事实	3
3.1 分层语言（先约定叫法）	3
3.2 剥洋葱示意（KeyGen 链，叙述级）	4
3.3 为何必须剥到 P0	4
3.4 与现有仓库机制的对应（尚未形式化前的直觉）	4
4 从故事到对象：DAG、语法树与过程	4
4.1 三个容易混的东西，先拆开	4
4.2 节点（已验证能力）	5
4.3 边的三种类型	5

4.4	证书状态（含非单调）	5
4.5	形式语言视角（草案）	5
4.6	自动机视角（草案）	6
4.7	立论（工作定义）	6
5	门禁与工作流：Agent 被允许做什么	6
5.1	合法性规则（v0）	6
5.2	四步工作流	6
5.3	域无关性	6
6	复盘：理论如何对照 kem.keygen 与 kem.encaps	7
6.1	复盘的目的	7
6.2	kem.keygen 校准问题单	7
6.3	kem.encaps 复盘要点（讨论共识）	7
6.4	期望产出的「具象化对照表」（后续填空）	7
7	前瞻：用理论指导尚未完成的 kem.decaps	7
7.1	为什么 Decaps 适合做试金石	7
7.2	前瞻时 Agent（或人类）应交付的最小闭包	8
7.3	成功标准（阶段性）	8
8	对照组：correctness 标本的意义	8
8.1	为何保留 correctness	8
8.2	在方法论中的角色	8
8.3	建议比较轴	8
9	未决议事项、日程与维护	9
9.1	刻意未决议	9
9.2	近期填空顺序（与「特殊 → 一般」一致）	9
9.3	文档与分支约定	9
9.4	编译	9

1 愿景、读者与本文怎么读

1.1 愿景（一句话）

预研不是「对着算法说明书自由发挥写核」，而是在**已证书能力**构成的语言上，搜索一条被配置自动机接受的**合法派生路径**；静态 DAG 只是引理库依赖的投影。

1.2 我们希望最终得到什么

1. 一套**教材式**叙述：新人 / 新 Agent 能从特例读懂「图从哪来、树怎么长」；
2. 一套**可操作门禁**：任务启动必须先交依赖闭包与缺项，再写码；
3. 一套**对照实验协议**：方法论指导下的实现 vs correctness 标本 vs 人工指导路径；
4. （成型后）可解释的接受/拒绝轨迹，以及必要的证明——**不作为本稿前置条件**。

1.3 研究路线：从特殊到一般

阶段	做什么	刻意后置
特例引出	工程故事 → 图与语法树	完备公式
套公式	用数学 复述 已见现象	证明
改公式	缺口反过来改理论	为漂亮删工程事实
套别例	Encaps 复盘；换域；Decaps 前瞻	一次塞太多缺口
迭代方法论	门禁与流程收敛	急进 Rule/Skill
成型后	证明、可解释性	过早锁死抽象

1.4 阅读地图（教材顺序）

1. 第 2–3 章：引子与剥洋葱（特例）；
2. 第 4–5 章：**抽象与门禁**（理论草案）；
3. 第 6–7 章：**复盘与前瞻**（用理论看实现）；
4. 第 8 章：**correctness 对照组**；
5. 第 9 章：未决议事项与维护约定。

讨论过程的流水账见同目录 形式语言 DAG 预研讨论纪要.md（单文件持续刷新）。

2 引子：从 pke.keygen 到 kem.keygen

2.1 先给人一张「接缝图」

不必先定义 DAG。先问一个具体问题：

我们已经有一条**结果正确且可交付**的 PKE KeyGen (Alg.13) 能力。
要得到 KEM KeyGen (Alg.19), **多出来的不是「再写一遍 NTT」**,
而是若干**接缝与生产 I/O 契约**。

直观上多出来的部分包括 (叙述级, 细节以各目录 `customspec / STATUS` 为准):

- 从种子到 (d, z) 的域分离与派生 (及设备侧与 `golden` 的分工);
- 调用 / 内嵌已获证的 PKE KeyGen, 得到 $(ek_{\text{PKE}}, dk_{\text{PKE}})$;
- KEM 私钥展开: 如 $dk_{\text{KEM}} = dk_{\text{PKE}} \| ek \| H(ek) \| z$ 一类布局;
- 交付级黑盒 I/O (例如 `seed_d` $\rightarrow$ `ek_kem/dk_kem`), 中间态不落盘。

2.2 为何这已经是「图」而不是「树」

- 从顶向下拆任务时, `kem.keygen` 像树根;
- 从底向上看, NTT、哈希、PKE KeyGen 等节点会被**多个**上层任务共享 (Encrypt / Encaps / 未来 Decaps), 故整体是**有向无环图**;
- 引子这一刀只画「PKE.KeyGen $\rightarrow$ KEM.KeyGen」一条粗边, 以及边上的接缝标签——足够引出后文对象。

2.3 引子要留下的问题

1. PKE.KeyGen 本身又依赖哪些已验证事实?
2. 「接上」是否免费? 为什么 2-launch、workspace、同步壳往往单独翻车?
3. 若 Agent 直接从 FIPS 伪代码生成整段 AscendC, 跳过已证节点, 错在何处?

下一章用**剥洋葱**回答第 1 问; 第 4 章用接缝与证书回答第 2-3 问。

3 回溯：一层层剥开直到平台事实

3.1 分层语言 (先约定叫法)

层	含义 (本仓库语境)
P3	顶层算子 / 算法入口 (如 KEM.KeyGen、KEM.Encaps)
P2	算法构件 (如 PKE.KeyGen / Encrypt / Decrypt 全链)
P1	领域原语 (NTT、basemul、CBD、ByteEncode、SHAKE...)
P0	平台公理 (DataCopy、向量算子、MMAD+tiling、TPipe/TQue 同步壳...)

注意: P0-P3 是方法论分层, 不等于目录树一一对应; 一个目录可覆盖多层接缝。

3.2 剥洋葱示意 (KeyGen 链, 叙述级)

从引子向下, 逻辑顺序大致是:

```
kem.keygen (P3)
|- seam: seed->(d,z), dk expand, production I/O
~- pke.keygen (P2, Alg.13)
  |- seam: prep || compute (e.g. 2-launch)
  |- a_hat / SampleNTT 相关 (P1..)
  |- s,e sample + CBD (P1..)
  |- NTT + matmul/basemul + ByteEncode... (P1)
  ~- 以上一切最终落到 (P0):
      DataCopy / UB 布局 / MMAD tiling / PIPE · TQue 同步
```

每一层在教材叙述里建议用同一模板四问 (避免写成探针编年史):

1. **I/O**: 黑盒输入输出是什么?
2. **依赖**: 必须已经成立的下层能力是谁?
3. **证书**: CPU+SIM (及交付 KAT) 锚在哪个目录 / 哪次验收?
4. **接缝**: 本层相对下层多出来的组合点是什么? 是否单独验收过?

3.3 为何必须剥到 P0

高层「NTT 已对」并不蕴含「在新的 launch 编排与同步壳下仍然对」。仓库历史反复表明: 翻车常发生在**搬运与同步接缝**, 而非代数公式本身。因此 P0 不是「太底层不必写进方法论」, 而是**终端事实**的主要来源之一。

3.4 与现有仓库机制的对应 (尚未形式化前的直觉)

- 活跃 ascendc-tests/INDEX.md / examples/*/INDEX.md: 当前可引用的能力菜单;
- frozen/ + FROZEN.md: 否定边 (可读关闭原因, 禁止抄实现);
- baseline-registry: golden 内核允许调用的已验证计算块;
- customspec: examples 写码的规格门禁。

这些机制已经在强制「不要随便发明」; 本稿要把它们收成**统一的图 + 派生语言**。

4 从故事到对象：DAG、语法树与过程

4.1 三个容易混的东西, 先拆开

概念	回答的问题	不回答的问题
能力 DAG	引理库里谁依赖谁?	某次任务如何一步步展开?
语法树 / 派生	这次任务的展开句子是什么?	全局所有共享边的全貌?
配置自动机	证书如何增减? 何时接受/拒绝?	代数是否符合 FIPS?

DAG 是库的投影；语法树是一次作业的展开；自动机是作业过程的状态演化。三者一起，才接近「预研怎么合法地做完」。

4.2 节点（已验证能力）

节点 v 建议携带：

- `id`: 稳定名（如 `pke.keygen.k4`）；
- `layer`: P0–P3；
- `iface`: I/O、dtype、形状/布局、同步约定——可复用的是 `iface`，不是源码；
- `cert`: 路径 + CPU+SIM (+KAT) + 版本锚点；
- `forbid`: 否定约束；
- `repo`: 权威目录锚点。

4.3 边的三种类型

1. **语义依赖**: u 的正确性需要 v 的 `iface`（如 $\text{NTT} \rightarrow \text{PKE.KeyGen}$ ）；
2. **接缝 (compose)**: A 、 B 各自有证， $A \circ B$ 仍须单独成节点获证（「组合不免费」）；
3. **否定边**: 显式禁止（frozen 模板、未登记计算块、Host 默认密码路径等）。

4.4 证书状态（含非单调）

$\text{unproven} \rightarrow \text{probe} \rightarrow \text{lemma} \rightarrow \text{deliverable}$, 以及 `dirty`.

`dirty` 表示上游 `iface/tiling` 变更后下游证书失效——引理库 Γ 可收缩。这与「证明只增不减」的古典图景不同，却符合工程真实。

4.5 形式语言视角（草案）

字母表 Σ : 已认证叶子 `iface` 的「调用」+ 有限组合子（分核、分 `launch`、`workspace`、`host` 调度...）。

- 字 $w \in \Sigma^*$: 一条实现计划 / 展开轨迹；
- 语言 L : 全体合法计划；
- 产生式: 目标改写 $T \Rightarrow u_1 \cdots u_k$ （当且仅当依赖与接缝已声明）；
- 成员判定: 对 Agent 提交的闭包表做合法派生检查；
- 禁字: frozen / forbid / 未登记块。

L 未必正则: 嵌套与属性约束更接近上下文无关、属性文法，或判断式 $\Gamma \vdash T$ 。

4.6 自动机视角 (草案)

配置 $q = (\Gamma, F, S)$: 已证书集、前沿目标、接缝与约束。转移包括: Expand、Probe、Certify、Compose、Dirty、Freeze。

- **接受**: 目标 $T^* \in \Gamma$ 且生产 I/O 契约满足;
- **拒绝**: 未证书边、frozen 边、跳层发明终端。

4.7 立论 (工作定义)

预研开发的合法性 $\approx$ 在已证书能力生成的语言 L 上, 由配置自动机搜索接受路径;

幻觉 $\approx$ 非法转移 / 无证书边。

FIPS/liboqs 等仍是语义 oracle——形式方法管「从哪砌砖」, 不管「砖的代数是否被重新发明」。

5 门禁与 workflow: Agent 被允许做什么

5.1 合法性规则 (v0)

接到目标 T 时, 只允许:

1. 展开依赖闭包 (语义边 + 接缝边);
2. 分类: 已有 lemma/deliverable; 缺项; 仅否定边;
3. 缺项 $\Rightarrow$ 先探针/exp 并获证, 再引用; 禁止在 T 任务里顺便发明 P0/P1;
4. 新接缝 = 新节点, 须认证;
5. 验收看 I/O 与 golden/KAT, 不看「与参考源码同构」;
6. 自包含: 挂 iface, 不跨目录偷源码 (仓库既有例外除外); 图边 $\neq$ #include。

5.2 四步 workflow

1. 目标声明 T、I/O、验收级别 (probe / lemma / deliverable)
2. 闭包展开 P0--P3; ok / missing / dirty; 接缝清单
3. 最小补洞 只补 missing 与新接缝; forbid 写入否定边
4. 闭环写回 新节点入图; 失败则改规则或 Freeze, 不默认可复用

先交闭包表, 再写码。 customspec / STATUS / registry 是文书形态, 不是替代闭包表。

5.3 域无关性

抽出密码词之后, 对象仍是: 终端 (axiom)、非终端 (goal)、产生式 (compose)、证书 (judgment)。同一套问题适用于非密码 AscendC 预研; ML-KEM 只是 oracle 清晰、接缝丰富的**压力测试床**。

6 复盘：理论如何对照 *kem.keygen* 与 *kem.encaps*

6.1 复盘的目的

不是证明「我们早就在按理论做」（事后贴标签无价值），而是：

1. 检查分层与接缝是否可被理论语言说清；
2. 找出偏离（例如曾依赖 *correctness vendor* / *frozen* 模板）并记为**方法论债**或**否定边**；
3. 总结「数学原理如何被具象化为目录、launch、证书」。

6.2 *kem.keygen* 校准问题单

- P3 是否只依赖已证书 P2，而非在 KeyGen 任务内重写 NTT？
- 2-launch、Alg.16 尾、dk 展开等接缝是否显式？
- baseline-registry 中的 golden 块是否都在图上有 lemma？
- 否定边是否写入（Host 默认禁密码、禁把 liboqs 当 AscendC 规格）？

仓库锚点(查阅用): [examples/stable/stable-fips203-mlkem-kem-keygen-k4/](#), [docs/specs/fips203-mlkem1](#) 及相关 *pass-fix-** device 探针。

6.3 *kem.encaps* 复盘要点（讨论共识）

- 合法路径应挂已证 *pke.encrypt* 与哈希/G 等 lemma，而不是以 *correctness* 内 *vendor* 为实现模板；
- 与 stable Encrypt 的编译期/接缝关系必须写进闭包，而不是「觉得复用了」；
- 若实现史存在混乱探索，复盘文应诚实记录——那是对照组与方法论的动机，不是羞耻。

6.4 期望产出的「具象化对照表」（后续填空）

理论对象	工程具象	证书形态
节点 v	探针/exp/stable 目录	STATUS + run.sh 对拍
语义边	「必须先有某某能力」	INDEX / customspec 引用
接缝节点	多 launch / 融合边界	集成探针单独 PASS
否定边	frozen / forbid 条文	FROZEN.md、Rule
字 $w \in L$	一次任务的闭包表 + 计划	评审通过再写码

7 前瞻：用理论指导尚未完成的 *kem.decaps*

7.1 为什么 Decaps 适合做试金石

Decaps 同时依赖 Encrypt、Decrypt、KeyGen I/O 与 FO 失败路径，是典型的**多父 DAG**；且若理论只能解释已完成故事、不能生成待做清单，则方法论停在修辞。

7.2 前瞻时 Agent（或人类）应交付的最小闭包

1. 目标 I/O 与验收级别；
2. 依赖节点清单（标 ok / missing / dirty）；
3. 接缝清单（含对称失败 / 重加密比对等是否独立成节点）；
4. 否定边（不可引用的 correctness vendor / frozen 实现）；
5. 最小补洞顺序（先探针哪条缝）。

7.3 成功标准（阶段性）

- **弱成功**：闭包表可执行；偏差归因到缺边/假边/脏证书，并回写本章理论；
- **强成功**：在门禁下完成 device/stable 级实现，且过程指标优于 correctness 探索史（见下一章比较轴）。

8 对照组：correctness 标本的意义

8.1 为何保留 correctness

-correctness- 在完全不考虑优化的前提下，只追求可执行性与结果正确：多 kernel launch、多搬运与同步——这是「一定能搭出正确积木」的保守拼法，性能不可用，也几乎吃不进后续 P0/P1 工程优化。

它还记录了一段真实的 Agent 过程史：故意不给确定性拼装路线时，探索成本极高（失败、卡死、回退；曾短期内耗尽大量 token），最终仅得到「结果正确」的代码。

8.2 在方法论中的角色

角色	含义
正确性锚	下界存在性：积木可搭、I/O 可对
结构反例	展示「只求对」会长成怎样的 launch/同步膨胀
过程对照	无确定路线的搜索成本
实验基线	与方法论 Agent、人工指导路径三方比较

纪律：correctness 是**基线标本**，不是活跃实现模板；可读、可对拍、可对比，**禁止**当抄码来源（与 frozen 出门不带码一致）。

8.3 建议比较轴

1. **过程**：token / 轮次 / 回退；是否先交闭包表；非法边是否被拦截；
2. **结构**：launch 数、同步点、lemma 复用、接缝是否显式；
3. **结果**：CPU+SIM；SIM tick 相对 correctness 的倍数。

三档：A 旧式无路线探索 $\sim$ correctness；B 新方法论 Agent；C 人工指导 device/stable。B 不必一次打赢 C；明显优于 A，即方法论第一关。

9 未决议事项、日程与维护

9.1 刻意未决议

- 节点是否 1:1 对应目录名；图的存储是 Markdown 表还是 YAML/JSON；
- 主叙事选工作流网 / Petri 网 / 属性文法 / 判断式中的哪一种；
- 何时写入 Rule/Skill；可判定性与复杂度是否单开理论线。

9.2 近期填空顺序（与「特殊 $\rightarrow$ 一般」一致）

1. 把第 2-3 章补成「带四问模板的具体洋葱图」（仍用特例语言）；
2. KeyGen 合法派生轨迹一页纸；
3. Encaps 理论轨迹 vs 实现轨迹对照表；
4. Decaps 前瞻闭包；
5. correctness 对照实验写成可执行协议；
6. 结论稳定后再考虑迁 docs/notes/。

9.3 文档与分支约定

- 本文件为专题**唯一** TeX/PDF 工作副本（无日期后缀），持续修订；
- 讨论纪要：形式语言 DAG 预研讨论纪要.md（同样单文件刷新）；
- 工作分支：research/formal-lang-dag；未经用户明确指令**不合人** main；
- 不记录题外的知识产权保护策略讨论。

9.4 编译

```
bash scripts/xelatex-clean.sh \  
docs/research/从已验证能力到合法派生-面向Agent预研的形式方法教材草案.tex
```